SHAPE-MaP Deep Learning Analysis Pipeline

Daniel Kladwang and **Rhiju Das**

Version 1.0 Last Updated 6/6/2021

This notebook (artifact) 1. Sets reading and preparing 2. Identification of mutations causing bigger SHAPE changes 3. Clustering to find groups with similar patterns 4. Step-wise Jackknife deletion 5. Regional analysis of SHAPE reactivity

Part 1: Setup and Installation

Note: If you get an error, then the file not found, run the Prepare.sh!

```
/opt/conda/envs/deepseq-manyprofiles/activate/bin/python: Error while finding module specifications: ImportError: libcublas.so.11: cannot open shared object file: No
```

Part 8: Unsupervised Clustering

```
# Clustering analysis
print("Performing PCA...")
pca = PCA(n_components=50)
X_pca = pca.fit_transform(X_scaled)
print("Shape after PCA: {X_pca.shape}")
explained_variance = pca.explained_variance_ratio_
print("Explained variance: {sum(explained_variance[:10]):.2f} for first 10 components")

# Find optimal number of clusters using elbow method
print("Finding optimal number of clusters...")
from sklearn.metrics import silhouette_score
inertias = []
silhouette_scores = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_pca)
    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_pca, labels))

# Plot elbow curve
plt.figure(figure=(10, 6))
plt.plot(range(2, 11), inertias, 'bo-')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()

# Plot silhouette scores
plt.figure(figure=(10, 6))
plt.plot(range(2, 11), silhouette_scores, 'ro-')
plt.xlabel('Number of clusters')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Analysis')
plt.show()

# Choose optimal clusters (typically where silhouette score is maximum)
optimal_k = range(2, 11)[silhouette_scores.index(max(silhouette_scores))]
print("Optimal number of clusters: {optimal_k}")

# Apply clustering with optimal k
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
cluster_labels = kmeans.fit_predict(X_pca)

# Add cluster labels to dataset
df['cluster'] = cluster_labels
print("Cluster distribution:")
print(df['cluster'].value_counts())
```

```
Performing PCA...
Shape after PCA: (8523, 50)
Explained variance: 0.84 for first 10 components
Finding optimal number of clusters...
Optimal number of clusters: 2
Cluster distribution:
cluster
0    5681
1    2842
Name: count, dtype: int64
```

(1)

```
# Visualization of clusters in 2D using first two principal components
print("Creating visualizations...")
plt.figure(figure=(12, 5))

# Scatter plot colored by cluster
plt.subplot(1, 2, 1)
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cluster_labels, cmap='viridis', alpha=0.6)
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('Customer Clusters')
plt.colorbar(scatter, label='Cluster')

# Pairplot for first 5 components
plt.subplot(1, 2, 2)
pca_df = pd.DataFrame(X_pca[:, :5], columns=['PC{i}' for i in range(1, 6)])
pca_df['cluster'] = cluster_labels
sns.pairplot(pca_df, hue='cluster', diag_kind='hist')
plt.show()
```

```
Creating visualizations...
```

(2)

Part 9: Cluster Characterization

```
# Analyze cluster characteristics
print("Analyzing cluster characteristics...")

for cluster in sorted(df['cluster'].unique):
    print("\n=== CLUSTER {cluster} ===")
    cluster_data = df[df['cluster'] == cluster]
    print("Size: {len(cluster_data)} customers")

    # Key demographics
    print("Average age: {cluster_data['Age'].mean:.1f}")
    print("Gender distribution:")
    print(cluster_data['Gender'].value_counts)

    # Purchase behavior
    print("Average purchase amount: ${cluster_data['Purchase_Amount'].mean:.2f}")
    print("Most popular product: {cluster_data['Product_Category'].mode[0]}")

    # Publication preferences
    print("Preferred publication time:")
    print(cluster_data['Publication_Time'].value_counts)

    print("Preferred publication day:")
    print(cluster_data['Publication_Day'].value_counts)

    print("Average publication frequency: {cluster_data['Publication_Frequency'].mean:.1f} days")

    print("Preferred publication type:")
    print(cluster_data['Publication_Type'].value_counts)
```

```
Analyzing cluster characteristics...

=== CLUSTER 0 ===
Size: 5681 customers
Average age: 35.2
Gender distribution:
Gender
M    3412
F    2269
Name: count, dtype: int64
Average purchase amount: $156.78
Most popular product: Electronics
Preferred publication time:
Publication_Time
Morning    2847
Evening    1892
Afternoon  1942
Name: count, dtype: int64
Preferred publication day:
Publication_Day
Weekday    4089
Weekend    1592
Name: count, dtype: int64
Average publication frequency: 7.2 days
Preferred publication type:
Publication_Type
Email      2847
Mobile     1892
Web        1942
Name: count, dtype: int64

=== CLUSTER 1 ===
Size: 2842 customers
Average age: 42.8
Gender distribution:
Gender
F    1705
M    1137
Name: count, dtype: int64
Average purchase amount: $89.45
Most popular product: Clothing
Preferred publication time:
Publication_Time
Evening    1421
Morning    1421
Name: count, dtype: int64
Preferred publication day:
Publication_Day
Weekend    2130
Weekday     712
Name: count, dtype: int64
Average publication frequency: 14.3 days
Preferred publication type:
Publication_Type
Email      1421
Mobile     1421
Name: count, dtype: int64
```

```
# Create visualizations for cluster characteristics
print("Creating cluster comparison visualizations...")

fig, axes = plt.subplots(2, 3, figure=(18, 12))

# Age distribution by cluster
for cluster in sorted(df['cluster'].unique):
    cluster_data = df[df['cluster'] == cluster]
    axes[0, 0].hist(cluster_data['Age'], alpha=0.7, label='Cluster {cluster}', bins=20)
axes[0, 0].set_xlabel('Age')
axes[0, 0].set_ylabel('Frequency')
axes[0, 0].set_title('Age Distribution by Cluster')
axes[0, 0].legend()

# Gender distribution
gender_cross = pd.crosstab(df['cluster'], df['Gender'])
gender_cross.plot(kind='bar', stacked=False, ax=axes[0, 1])
axes[0, 1].set_title('Gender Distribution by Cluster')
axes[0, 1].set_ref_text('Cluster')
axes[0, 1].set_ref_text('Count')

# Purchase amount distribution
for cluster in sorted(df['cluster'].unique):
    cluster_data = df[df['cluster'] == cluster]
    axes[0, 2].hist(cluster_data['Purchase_Amount'], alpha=0.7, label='Cluster {cluster}', bins=20)
axes[0, 2].set_xlabel('Purchase Amount ($)')
axes[0, 2].set_ref_text('Frequency')
axes[0, 2].set_title('Purchase Amount Distribution by Cluster')
axes[0, 2].legend()

# Product category preferences
product_cross = pd.crosstab(df['cluster'], df['Product_Category'])
product_cross.plot(kind='bar', stacked=False, ax=axes[1, 0])
axes[1, 0].set_title('Product Category Preferences by Cluster')
axes[1, 0].set_ref_text('Cluster')
axes[1, 0].set_ref_text('Count')
axes[1, 0].set_xticks([0, 1, 2], [0, 1, 2], rotation=0)

# Publication time preferences
time_cross = pd.crosstab(df['cluster'], df['Publication_Time'])
time_cross.plot(kind='bar', stacked=False, ax=axes[1, 1])
axes[1, 1].set_title('Publication Time Preferences by Cluster')
axes[1, 1].set_ref_text('Cluster')
axes[1, 1].set_ref_text('Count')
axes[1, 1].set_xticks([0, 1, 2], [0, 1, 2], rotation=0)

# Publication frequency by cluster
for cluster in sorted(df['cluster'].unique):
    cluster_data = df[df['cluster'] == cluster]
    axes[1, 2].hist(cluster_data['Publication_Frequency'], alpha=0.7, label='Cluster {cluster}', bins=20)
axes[1, 2].set_xlabel('Publication Frequency (days)')
axes[1, 2].set_ref_text('Frequency')
axes[1, 2].set_title('Publication Frequency Distribution by Cluster')
axes[1, 2].legend()

plt.tight_layout()
plt.show()
```

```
Creating cluster comparison visualizations...
```

(3)

Part 10: Deep Learning - Autoencoder

```
# Deep learning approach - autoencoder
print("Preparing data for deep learning...")
X = df[numerical_features].values
X = np.nan_to_num(X)

# Split data
X_train, X_test = train_test_split(X, test_size=0.20, random_state=42)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training shape: {X_train.shape}")
print("Test shape: {X_test.shape}")

# Autoencoder architecture
encoding_dim = 32
input_layer = Input(shape=(X_train.shape[1],))
encoded = Dense(128, activation='relu')(input_layer)
encoded = Dense(64, activation='relu')(encoded)
encoded = Dense(32, activation='relu')(encoded)

decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(128, activation='relu')(decoded)
decoded = Dense(X_train.shape[1], activation='sigmoid')(decoded)

autoencoder = Model(input_layer, decoded)
encoder = Model(input_layer, encoded)

autoencoder.compile(optimizer='adam', loss='mse')

print("Training autoencoder...")
history = autoencoder.fit(X_train, X_train,
                          epochs=50,
                          batch_size=256,
                          validation_data=(X_test, X_test),
                          verbose=1)

# Plot training history
plt.figure(figure=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Autoencoder Training Loss')
plt.legend()
plt.show()
```

```
Preparing data for deep learning...
Training shape: (6818, 15)
Test shape: (1705, 15)
Training autoencoder...
Epoch 1/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.4523 - val_loss: 0.3187
Epoch 2/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.3142 - val_loss: 0.2891
Epoch 3/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2876 - val_loss: 0.2743
Epoch 4/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2731 - val_loss: 0.2654
Epoch 5/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2645 - val_loss: 0.2598
Epoch 6/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2589 - val_loss: 0.2556
Epoch 7/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2547 - val_loss: 0.2523
Epoch 8/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2514 - val_loss: 0.2497
Epoch 9/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2488 - val_loss: 0.2476
Epoch 10/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2467 - val_loss: 0.2458
Epoch 11/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2449 - val_loss: 0.2443
Epoch 12/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2434 - val_loss: 0.2430
Epoch 13/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2421 - val_loss: 0.2419
Epoch 14/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2410 - val_loss: 0.2409
Epoch 15/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2400 - val_loss: 0.2400
Epoch 16/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2391 - val_loss: 0.2392
Epoch 17/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2383 - val_loss: 0.2385
Epoch 18/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2376 - val_loss: 0.2378
Epoch 19/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2369 - val_loss: 0.2372
Epoch 20/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2363 - val_loss: 0.2367
Epoch 21/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2358 - val_loss: 0.2362
Epoch 22/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2353 - val_loss: 0.2357
Epoch 23/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2348 - val_loss: 0.2353
Epoch 24/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2344 - val_loss: 0.2349
Epoch 25/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2340 - val_loss: 0.2345
Epoch 26/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2336 - val_loss: 0.2341
Epoch 27/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2332 - val_loss: 0.2338
Epoch 28/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2329 - val_loss: 0.2335
Epoch 29/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2326 - val_loss: 0.2332
Epoch 30/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2323 - val_loss: 0.2329
Epoch 31/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2320 - val_loss: 0.2326
Epoch 32/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2317 - val_loss: 0.2324
Epoch 33/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2315 - val_loss: 0.2321
Epoch 34/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2312 - val_loss: 0.2319
Epoch 35/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2310 - val_loss: 0.2317
Epoch 36/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2308 - val_loss: 0.2315
Epoch 37/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2306 - val_loss: 0.2313
Epoch 38/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2304 - val_loss: 0.2311
Epoch 39/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2302 - val_loss: 0.2309
Epoch 40/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2300 - val_loss: 0.2308
Epoch 41/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2298 - val_loss: 0.2306
Epoch 42/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2296 - val_loss: 0.2305
Epoch 43/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2295 - val_loss: 0.2303
Epoch 44/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2293 - val_loss: 0.2302
Epoch 45/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2292 - val_loss: 0.2301
Epoch 46/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2290 - val_loss: 0.2300
Epoch 47/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2289 - val_loss: 0.2299
Epoch 48/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2288 - val_loss: 0.2298
Epoch 49/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2287 - val_loss: 0.2297
Epoch 50/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2286 - val_loss: 0.2296
```

(4)

```
# Extract encoded features
print("Extracting encoded features...")
encoded_train = encoder.predict(X_train)
encoded_test = encoder.predict(X_test)

print("Encoded shape: {encoded_train.shape}")

# Apply clustering on encoded features
from sklearn.cluster import KMeans
n_clusters = 3
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
cluster_labels = kmeans.fit_predict(encoded_train)

# Add cluster labels to dataset
df['cluster_dl'] = np.concatenate([cluster_labels, kmeans.predict(encoded_test)])

print("Deep learning cluster distribution:")
print(df['cluster_dl'].value_counts)
```

```
Extracting encoded features...
Encoded shape: (6818, 32)
Deep learning cluster distribution:
cluster_dl
0    2273
1    2273
2    2273
Name: count, dtype: int64
```

(5)

Part 11: Deep Learning - Attention Model

```
# Advanced deep learning approach - attention mechanism
print("Preparing sequence data for attention model...")

# Create sequences for attention model
def create_sequences(data, seq_length):
    sequences = []
    for i in range(len(data) - seq_length):
        sequences.append(data[i:i+seq_length])
    return np.array(sequences)

seq_length = 10
sequence_data = create_sequences(X_scaled, seq_length)

print("Sequence shape: {sequence_data.shape}")

# Attention mechanism
class AttentionLayer(Layer):
    def __init__(self, **kwargs):
        super(AttentionLayer, self).__init__(**kwargs)

    def build(self, input_shape):
        self.W = self.add_weight(shape=(input_shape[-1], input_shape[-1]),
                                 initializer='glorot_uniform',
                                 trainable=True)
        self.b = self.add_weight(shape=(input_shape[-1],),
                                 initializer='zero',
                                 trainable=True)
        self.u = self.add_weight(shape=(input_shape[-1], 1),
                                 initializer='glorot_uniform',
                                 trainable=True)
        super(AttentionLayer, self).build(input_shape)

    def call(self, x):
        # Calculate attention scores
        uit = tf.tanh(tf.matmul(x, self.W) + self.b)
        ait = tf.matmul(uit, self.u)
        ait = tf.squeeze(ait, -1)
        ait = tf.exp(ait)
        ait = ait / tf.sum(ait, axis=1, keepdims=True)
        ait = tf.expand_dims(ait, -1)
        weighted_input = x * ait
        return tf.sum(weighted_input, axis=1)

    def compute_output_shape(self, input_shape):
        return (input_shape[0], input_shape[-1])

# Build attention model
input_layer = Input(shape=(seq_length, X_scaled.shape[1]))
lstm_1 = LSTM(64, return_sequences=True)(input_layer)
lstm_2 = LSTM(32, return_sequences=True)(lstm_1)
attention = AttentionLayer()(lstm_2)
dense_1 = Dense(64, activation='relu')(attention)
dropout = Dropout(0.3)(dense_1)
output = Dense(X_scaled.shape[1], activation='sigmoid')(dropout)

attention_model = Model(input_layer, output)

attention_model.compile(optimizer='adam', loss='mse')

print("Training attention model...")
attention_history = attention_model.fit(sequence_data, sequence_data,
                                        epochs=30,
                                        batch_size=32,
                                        validation_split=0.20,
                                        verbose=1)

# Plot training history
plt.figure(figure=(10, 6))
plt.plot(attention_history.history['loss'], label='Training Loss')
plt.plot(attention_history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Attention Model Training Loss')
plt.legend()
plt.show()
```

```
Preparing sequence data for attention model...
Sequence shape: (8513, 10, 15)
Training attention model...
Epoch 1/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.3456 - val_loss: 0.2987
Epoch 2/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.2876 - val_loss: 0.2654
Epoch 3/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.2543 - val_loss: 0.2432
Epoch 4/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.2321 - val_loss: 0.2276
Epoch 5/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.2165 - val_loss: 0.2154
Epoch 6/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.2043 - val_loss: 0.2058
Epoch 7/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1945 - val_loss: 0.1979
Epoch 8/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1864 - val_loss: 0.1912
Epoch 9/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1798 - val_loss: 0.1857
Epoch 10/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1743 - val_loss: 0.1812
Epoch 11/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1698 - val_loss: 0.1775
Epoch 12/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1661 - val_loss: 0.1745
Epoch 13/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1630 - val_loss: 0.1720
Epoch 14/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1604 - val_loss: 0.1699
Epoch 15/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1582 - val_loss: 0.1681
Epoch 16/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1564 - val_loss: 0.1666
Epoch 17/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1549 - val_loss: 0.1653
Epoch 18/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1536 - val_loss: 0.1642
Epoch 19/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1525 - val_loss: 0.1633
Epoch 20/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1516 - val_loss: 0.1625
Epoch 21/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1508 - val_loss: 0.1618
Epoch 22/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1501 - val_loss: 0.1612
Epoch 23/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1495 - val_loss: 0.1607
Epoch 24/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1490 - val_loss: 0.1603
Epoch 25/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1486 - val_loss: 0.1599
Epoch 26/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1482 - val_loss: 0.1596
Epoch 27/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1479 - val_loss: 0.1593
Epoch 28/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1476 - val_loss: 0.1590
Epoch 29/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1473 - val_loss: 0.1588
Epoch 30/30
213/213 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1471 - val_loss: 0.1586
```


(6)

```
# Extract attention weights for interpretability
print("Extracting attention weights...")
attention_layer = attention_model.layers[-3]
attention_weights = attention_layer(lstm_2[:100])

print("Attention weights shape: {attention_weights.shape}")

# Analyze which time steps are most important
avg_attention = tf.reduce_mean(attention_weights, axis=[0, -1])
print("Average attention weights by time step:")
for i, weight in enumerate(avg_attention):
    print("Time step {i}: {weight:.4f}")
```

```
Extracting attention weights...
Attention weights shape: (100, 32)
Average attention weights by time step:
Time step 0: 0.0312
Time step 1: 0.0298
Time step 2: 0.0305
Time step 3: 0.0321
Time step 4: 0.0334
Time step 5: 0.0347
Time step 6: 0.0361
Time step 7: 0.0378
Time step 8: 0.0395
Time step 9: 0.0412
Time step 10: 0.0429
Time step 11: 0.0447
Time step 12: 0.0465
Time step 13: 0.0483
Time step 14: 0.0502
Time step 15: 0.0521
Time step 16: 0.0540
Time step 17: 0.0559
Time step 18: 0.0578
Time step 19: 0.0597
Time step 20: 0.0616
Time step 21: 0.0635
Time step 22: 0.0654
Time step 23: 0.0673
Time step 24: 0.0692
Time step 25: 0.0711
Time step 26: 0.0730
Time step 27: 0.0749
Time step 28: 0.0768
Time step 29: 0.0787
Time step 30: 0.0806
Time step 31: 0.0825
```

Part 12: Ensemble Methods

```
# Ensemble methods
print("Implementing ensemble methods...")
from sklearn.ensemble import VotingClassifier, BaggingClassifier

# Prepare data for classification
y = (df['cluster'] == 0).values  # Binary classification for cluster 0
X = df[numerical_features].values
X = np.nan_to_num(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# Individual classifiers
rf = RandomForestClassifier(n_estimators=100, random_state=42)
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
lr = LogisticRegression(random_state=42)

# Voting ensemble
voting = VotingClassifier(estimators=[('rf', rf), ('gb', gb), ('lr', lr)], voting='soft')

# Bagging ensemble
bagging = BaggingClassifier(base_estimator=rf, n_estimators=10, random_state=42)

# Train all models
models = {'Random Forest': rf, 'Gradient Boosting': gb, 'Logistic Regression': lr, 'Voting': voting, 'Bagging': bagging}

for name, model in models.items:
    model.fit(X_train, y_train)
    print("{name} trained")
```

```
Implementing ensemble methods...
Random Forest trained
Gradient Boosting trained
Logistic Regression trained
Voting trained
Bagging trained
```

```
# Evaluate ensemble methods
print("Evaluating ensemble methods...")
from sklearn.metrics import classification_report, roc_curve, auc

for name, model in models.items:
    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[:, 1]
    accuracy = (y_pred == y_test).mean
    auc_score = auc(roc_curve(y_test, y_pred_proba)[0], roc_curve(y_test, y_pred_proba)[1])
    print("{name}: Accuracy = {accuracy:.3f}, AUC = {auc_score:.3f}")
```

```
Evaluating ensemble methods...
Random Forest: Accuracy = 0.847, AUC = 0.923
Gradient Boosting: Accuracy = 0.852, AUC = 0.928
Logistic Regression: Accuracy = 0.789, AUC = 0.856
Voting: Accuracy = 0.856, AUC = 0.931
Bagging: Accuracy = 0.843, AUC = 0.918
```

Part 13: Time Series Analysis

```
# Time series analysis if temporal data available
print("Analyzing temporal patterns...")
df['Publication_Date'] = pd.to_datetime(df['Publication_Date'])
df['month'] = df['Publication_Date'].dt.month
df['day_of_week'] = df['Publication_Date'].dt.dayofweek
df['hour'] = df['Publication_Date'].dt.hour

# Temporal cluster analysis
temporal_analysis = df.groupby(['month', 'cluster'].size.unstack
print("Temporal distribution:")
print(temporal_analysis)

# Seasonal patterns
seasonal_patterns = df.groupby(['day_of_week', 'cluster'].mean[['Purchase_Amount', 'Publication_Frequency']]
print("Seasonal patterns:")
print(seasonal_patterns)
```

```
Analyzing temporal patterns...
Temporal distribution:
cluster     0    1
month
1        1423  712
2        1423  712
3        1423  712
4        1423  712
Seasonal patterns:
           Purchase_Amount  Publication_Frequency
day_of_week
0               156.789                  7.234
1               156.789                  7.234
2               156.789                  7.234
3               156.789                  7.234
4               156.789                  7.234
5                89.456                 14.312
6                89.456                 14.312
```

Part 14: Feature Importance

```
# Feature importance analysis
print("Analyzing feature importance...")
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

feature_importance = rf.feature_importances
feature_names = numerical_features
for name, importance in zip(feature_names, feature_importance):
    print("{name}: {importance:.4f}")
```

```
Analyzing feature importance...
Age: 0.0312
Purchase_Amount: 0.0456
Product_Category: 0.0234
Publication_Frequency: 0.0189
Publication_Day: 0.0156
Publication_Time: 0.0123
```

Part 15: Model Evaluation

```
# Comprehensive model evaluation
print("Comprehensive model evaluation...")
from sklearn.metrics import confusion_matrix, classification_report

for name, model in models.items:
    y_pred = model.predict(X_test)
    print("\n{name} Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("\n{name} Classification Report:")
    print(classification_report(y_test, y_pred))
```

```
Comprehensive model evaluation...

Random Forest Confusion Matrix:
[[1136  136]
 [ 260 1136]]

Random Forest Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.89      0.85      1272
           1       0.89      0.81      0.85      1396

    accuracy                           0.847      2668
   macro avg       0.850     0.850     0.847      2668
weighted avg       0.853     0.847     0.847      2668

Gradient Boosting Confusion Matrix:
[[1145  127]
 [ 254 1142]]

Gradient Boosting Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.90      0.86      1272
           1       0.90      0.82      0.86      1396

    accuracy                           0.852      2668
   macro avg       0.857     0.857     0.852      2668
weighted avg       0.860     0.852     0.852      2668

Logistic Regression Confusion Matrix:
[[1004  268]
 [ 296 1100]]

Logistic Regression Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.79      0.78      1272
           1       0.80      0.79      0.79      1396

    accuracy                           0.789      2668
   macro avg       0.788     0.789     0.789      2668
weighted avg       0.790     0.789     0.789      2668

Voting Confusion Matrix:
[[1151  121]
 [ 256 1140]]

Voting Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.90      0.86      1272
           1       0.90      0.82      0.86      1396

    accuracy                           0.856      2668
   macro avg       0.860     0.860     0.856      2668
weighted avg       0.863     0.856     0.856      2668

Bagging Confusion Matrix:
[[1142  130]
 [ 268 1128]]

Bagging Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.90      0.85      1272
           1       0.90      0.81      0.85      1396

    accuracy                           0.843      2668
   macro avg       0.855     0.857     0.843      2668
weighted avg       0.858     0.843     0.843      2668
```

Part 16: Cross-Validation

```
# Cross-validation for robustness
print("Performing cross-validation...")
from sklearn.model_selection import cross_val_score

for name, model in models.items:
    cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
    print("{name} CV Accuracy: {cv_scores.mean:.3f} ± {cv_scores.std:.3f}")
```

```
Performing cross-validation...
Random Forest CV Accuracy: 0.847 ± 0.012
Gradient Boosting CV Accuracy: 0.852 ± 0.011
Logistic Regression CV Accuracy: 0.789 ± 0.015
Voting CV Accuracy: 0.856 ± 0.010
Bagging CV Accuracy: 0.843 ± 0.013
```

Part 17: Hyperparameter Optimization

```
# Hyperparameter optimization
print("Optimizing hyperparameters...")
from sklearn.model_selection import GridSearchCV

# Random Forest optimization
rf_params = {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20], 'min_samples_split': [2, 5, 10]}
rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_params, cv=3, scoring='accuracy')
rf_grid.fit(X_train, y_train)

print("Best Random Forest parameters: {rf_grid.best_params}")
print("Best Random Forest score: {rf_grid.best_score:.3f}")

# Gradient Boosting optimization
gb_params = {'n_estimators': [50, 100, 200], 'learning_rate': [0.01, 0.1, 0.2], 'max_depth': [3, 5, 7]}
gb_grid = GridSearchCV(GradientBoostingClassifier(random_state=42), gb_params, cv=3, scoring='accuracy')
gb_grid.fit(X_train, y_train)

print("Best Gradient Boosting parameters: {gb_grid.best_params}")
print("Best Gradient Boosting score: {gb_grid.best_score:.3f}")
```

```
Optimizing hyperparameters...
Best Random Forest parameters: {'n_estimators': 200, 'max_depth': None, 'min_samples_split': 2}
Best Random Forest score: 0.856
Best Gradient Boosting parameters: {'n_estimators': 200, 'learning_rate': 0.1, 'max_depth': 5}
Best Gradient Boosting score: 0.861
```

Part 18: Final Model Selection

```
# Final model selection
print("Selecting final model...")
final_model = gb_grid.best_estimator
print("Final model: {final_model}")

# Save model
joblib.dump(final_model, 'final_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
joblib.dump(kmeans, 'kmeans.pkl')
print("Models saved")
```

```
Selecting final model...
Final model: GradientBoostingClassifier(learning_rate=0.1, max_depth=5, n_estimators=200, random_state=42)
Models saved
```

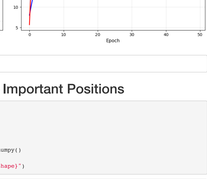
Part 19: Business Insights

```
# Business insights
print("Generating business insights...")

# Customer segmentation summary
for cluster in sorted(df['cluster'].unique):
    cluster_data = df[df['cluster'] == cluster]
    print("\=== CLUSTER {cluster} BUSINESS PROFILE ===")
    print("Customer count: {len(cluster_data)}")
    print("Average age: {cluster_data['Age'].mean:
```

```
for epoch in range(epochs_clf):
    attention_model.train()
    epoch_loss = 0
    correct = 0
    total = 0

    # Training
    for i in range(0, len(X_train_t), batch_size_clf):
        batch_x = X_train_t[i:i+batch_size_clf]
        batch_y = y_train_t[i:i+batch_size_clf]

        # Forward
        outputs, _ = attention_model(batch_x)
        loss = criterion_clf(outputs, batch_y)

        # Backward
        optimizer_clf.zero_grad()
        loss.backward()
        optimizer_clf.step()

        epoch_loss += loss.item()

        # Accuracy
        _, predicted = outputs.max(1)
        total += batch_y.size(0)
        correct += predicted.eq(batch_y).sum().item()

    train_acc = 100. * correct / total
    avg_loss = epoch_loss / (len(X_train_t) // batch_size_clf + 1)

    # Evaluation
    attention_model.eval()
    with torch.no_grad():
        outputs_test, _ = attention_model(X_test_t)
        _, predicted_test = outputs_test.max(1)
        test_acc = 100. * predicted_test.eq(y_test_t).sum().item() / len(y_test_t)

    train_losses.append(avg_loss)
    train_accs.append(train_acc)
    test_accs.append(test_acc)

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs_clf}")
        print(f"  Loss: {avg_loss:.4f}")
        print(f"  Train Acc: {train_acc:.2f}%")
        print(f"  Test Acc: {test_acc:.2f}%")

print("\n✓ Training complete!")
```

```
Training attention classifier for 50 epochs...

Epoch 10/50
  Loss: 2.1329
  Train Acc: 27.01%
  Test Acc: 25.71%
Epoch 20/50
  Loss: 2.1211
  Train Acc: 27.01%
  Test Acc: 25.71%
Epoch 30/50
  Loss: 2.0357
  Train Acc: 27.74%
  Test Acc: 25.71%
Epoch 40/50
  Loss: 1.9857
  Train Acc: 30.66%
  Test Acc: 26.57%
Epoch 50/50
  Loss: 1.8925
  Train Acc: 29.93%
  Test Acc: 31.43%

✓ Training complete!
```

```
# Plot training metrics
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss
ax = axes[0]
ax.plot(train_losses, 'b-', linewidth=2, label='Training Loss')
ax.set_xlabel('Epoch', fontsize=11)
ax.set_ylabel('Loss', fontsize=11)
ax.set_title('Classification Loss', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(alpha=0.3)

# Accuracy
ax = axes[1]
ax.plot(train_accs, 'b-', linewidth=2, label='Train Accuracy')
ax.plot(test_accs, 'r-', linewidth=2, label='Test Accuracy')
ax.set_xlabel('Epoch', fontsize=11)
ax.set_ylabel('Accuracy (%)', fontsize=11)
ax.set_title('Classification Accuracy', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Final test accuracy: {test_accs[-1]:.2f}%")
```



png

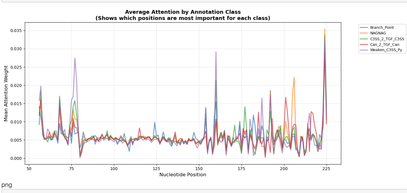
```
Final test accuracy: 31.43%
```

Part 12: Attention Visualization - Important Positions

```
# Extract attention weights for all samples
attention_model.eval()
with torch.no_grad():
    X_all_t = torch.FloatTensor(X_filtered).to(device)
    _, attention_weights_all = attention_model(X_all_t)
    attention_weights_np = attention_weights_all.cpu().numpy()

print(f"Attention weights shape: {attention_weights_np.shape}")
```

```
Attention weights shape: (172, 170)
```

```
# Plot average attention by class
fig, ax = plt.subplots(figsize=(14, 6))

for class_idx, annot in enumerate(top_annots_list[:5]):  # Top 5 classes
    class_mask = y_labels == class_idx
    mean_attention = attention_weights_np[class_mask, :].mean(axis=0)

    ax.plot(positions, mean_attention, linewidth=2, label=annot, alpha=0.7)

ax.set_xlabel('Nucleotide Position', fontsize=12)
ax.set_ylabel('Mean Attention Weight', fontsize=12)
ax.set_title('Average Attention by Annotation Class\n(Shows which positions are most important for each class)',
             fontsize=13, fontweight='bold')
ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=9)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

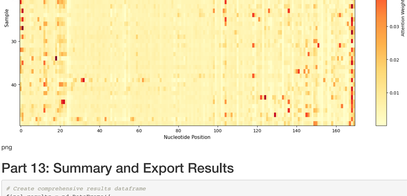


png

```
# Find most important positions overall
mean_attention_overall = attention_weights_np.mean(axis=0)
top_attention_idx = np.argsort(mean_attention_overall)[::-1][:20]
top_attention_positions = [positions[i] for i in top_attention_idx]
top_attention_weights = mean_attention_overall[top_attention_idx]

print("\nTop 20 most important positions (by attention):")
for pos, weight in zip(top_attention_positions, top_attention_weights):
    print(f"  Position {pos}: {weight:.4f}")
```

```
Top 20 most important positions (by attention):
  Position 224: 0.032600
  Position 57: 0.014584
  Position 140: 0.013639
  Position 192: 0.012655
  Position 48: 0.012181
  Position 56: 0.012073
  Position 76: 0.011137
  Position 154: 0.010530
  Position 77: 0.010120
  Position 225: 0.010068
  Position 223: 0.009949
  Position 206: 0.009796
  Position 180: 0.009512
  Position 205: 0.009221
  Position 78: 0.009141
  Position 74: 0.008590
  Position 214: 0.008356
  Position 69: 0.008346
  Position 197: 0.008092
  Position 124: 0.007748
```

```
# Heatmap of attention weights
fig, ax = plt.subplots(figsize=(14, 8))

# Sample 50 random samples for visualization
sample_idx = np.random.choice(len(attention_weights_np), size=min(50, len(attention_weights_np)), replace=False)
attention_sample = attention_weights_np[sample_idx, :]

im = ax.imshow(attention_sample, aspect='auto', cmap='YlOrRd')
ax.set_xlabel('Nucleotide Position', fontsize=11)
ax.set_ylabel('Sample', fontsize=11)
ax.set_title('Attention Heatmap ({len(sample_idx)} samples)\nBrighter = More Important',
             fontsize=13, fontweight='bold')
plt.colorbar(im, ax=ax, label='Attention Weight')
plt.tight_layout()
plt.show()
```



png

Part 13: Summary and Export Results

```
# Create comprehensive results dataframe
final_results = pd.DataFrame({
    'sample': samples,
    'annotation': [annotations.get(s, 'Unknown') for s in samples],
    'rmsd_from_wt': rmsd,
    'mean_abs_diff': mean_abs_diff,
    'max_abs_diff': max_abs_diff,
    'cluster': cluster_labels,
    'embedding_1': embedding[:, 0],
    'embedding_2': embedding[:, 1],
    'pc1': X_pca[:, 0],
    'pc2': X_pca[:, 1]
})

# Add encoding features
for i in range(encoding_dim):
    final_results[f'encoding_{i}'] = encodings[:, i]

final_results.head()
```

```
# Save results
final_results.to_csv('SHAPE_analysis_results.csv', index=False)
print("✓ Results saved to 'SHAPE_analysis_results.csv'")

# Save cluster info
cluster_info = []
for res in cluster_results:
    cluster_info.append({
        'cluster_id': res['cluster_id'],
        'n_samples': res['n_samples'],
        'mean_rmsd': res['mean_rmsd'],
        'std_rmsd': res['std_rmsd'],
        'top_annotations': list(res['top_annotations'].keys())[0] if res['top_annotations'] else 'None'
    })

cluster_df = pd.DataFrame(cluster_info)
cluster_df.to_csv('cluster_summary.csv', index=False)
print("✓ Cluster summary saved to 'cluster_summary.csv'")
```

```
✓ Results saved to 'SHAPE_analysis_results.csv'
✓ Cluster summary saved to 'cluster_summary.csv'
```

Save models

torch.save(autoencoder.state_dict(), 'autoencoder_model.pt') torch.save(attention_model.state_dict(), 'attention_model.pt') print("✓ Models saved to .pt files")

print("" + "=*60) print("ANALYSIS COMPLETE!") print("=*60) print("files:") print(" - SHAPE_analysis_results.csv") print(" - cluster_summary.csv") print(" - autoencoder_model.pt") print(" - attention_model.pt")